Version 1.1

January 2026

DATA STRUCTURES

MODULE 2 - ARRAY LIST & LINKED LIST

Author:

MUHAMMAD ILHAM PERDANA. S.Tr.T., M.T.

AHMAD NUR MU'MININ

WAHYU ANDIKA

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
INTRODUCTION **4**
OBJECTIVES 4
MODULE TARGETS 4
PREPARATION 4
KEYWORDS 4
CONTENTS **5**
1. Array List **5**
1.1. Definition of ArrayList 5
1) ArrayList() 5
2) ArrayList(Collection<? extends E> c) 5
3) ArrayList(int initialCapacity) 5
1.2. Characteristics of ArrayList 5
1.3. How ArrayList Works Internally 6
1.4. Important ArrayList Methods 7
1.5. Example Use Cases of ArrayList 8
1) Declaration ArrayList 8
2) add(element) 9
3) add(index, element) 9
4) get(index) 9
5) set(index,element) 10
6) remove(index) 10
7) size() 10
8) isEmpty() 11
9) clear() 11
2. Linked List **11**
2.1. Definition of LinkedList 11
2.2. Basic Structure of a LinkedList 11
1) Node 12
2) Data 12
3) Next & Previous Point 12
4) Pointer 13
6) Tail 13
2.3. Types of LinkedList 14
1) Singly Linked List 14
2) Doubly Linked List 14
3) Singly Circular Linked List 15
4) Doubly Circular Linked List 15
2.4. Important LinkedList Operations 16

1) Insertion.....16
2) Deletion.....16
3) Traversal.....16
4) Searching.....17
5) Updating.....17
6) Count Nodes.....17
2.5. Example Use Cases of LinkedList.....17
CODELAB.....25
Mission 1: Jujutsu Kaisen - ArrayList.....25
Mission 2: One Piece - LinkedList File:.....25
DEMO.....27
TASK 1 Passenger Management System (ArrayList).....27
Task 2: Train Schedule System (LinkedList).....28
Grading Rubric.....30
Grading Scale.....30

INTRODUCTION

OBJECTIVES

1. Understand the basic concepts of ArrayList and LinkedList and the differences in their internal structures.
2. Explain the characteristics of ArrayList and LinkedList.
3. Implement ArrayList and LinkedList in Java independently.
4. Use essential methods such as add, remove, get, set, traversal, search, update, and count.
5. Analyze and choose the appropriate data structure based on the needs of a given case study.
6. Create data management simulations using ArrayList and LinkedList (e.g., ticket booking systems, passenger queues, dynamic lists).
7. Build a visual understanding of how nodes, pointers, head, tail, and links between nodes work.

MODULE TARGETS

1. Explain the concept and differences between ArrayList and LinkedList as dynamic data structures in Java.
2. Identify the characteristics, strengths, and weaknesses of ArrayList and LinkedList for various use cases.
3. Implement ArrayList and LinkedList in Java programs using proper syntax and constructors.
4. Use essential methods (add, remove, get, set, size, clear, traversal, search, update) effectively in both data structures.
5. Visualize and describe how nodes, pointers, head, tail, and links work in a LinkedList.
6. Develop simple applications or simulations such as ticket booking systems or queue management using ArrayList or LinkedList.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. Java Development Kit
4. Java Runtime Environment
5. IDE (Intellij IDEA, Eclipse, Netbeans, etc.)
6. Strong motivation and commitment to learning

KEYWORDS

ArrayList, LinkedList, Node, Pointers, Head, Tail

CONTENTS

1. Array List

1.1. Definition of ArrayList

An `ArrayList` is a dynamic data structure in Java that works like a flexible version of an array. Unlike a regular array that has a fixed size, an ArrayList can automatically grow or shrink depending on how many elements are added or removed. It stores data in an ordered sequence and allows fast access to any element using an index. Java also provides many built-in methods for managing the data, such as adding, removing, or updating elements, making ArrayList easier to use than a traditional array. Internally, an `ArrayList` is implemented using a standard array that is automatically resized as needed, so programmers do not need to manage memory manually.

ArrayList in Java provides multiple constructors to create an ArrayList based on different requirements:

`1) ArrayList()`

Creates an empty ArrayList with default initial capacity.

```
ArrayList<Integer> arr = new ArrayList<>();
```

`2) ArrayList(Collection<? extends E> c)`

Creates an ArrayList initialized with elements from the specified collection.

```
ArrayList<String> arr = new ArrayList<>(collection);
```

`3) ArrayList(int initialCapacity)`

This constructor is used to build an array list with the initial capacity being specified.

```
ArrayList<Double> arr = new ArrayList<>(20);
```

1.2. Characteristics of ArrayList

An **ArrayList** has several characteristics that make it different from other data structures. First, all elements are stored **in order** and each element has a **clear index**, so we can read or change data very quickly. The ArrayList also has many **built-in methods**

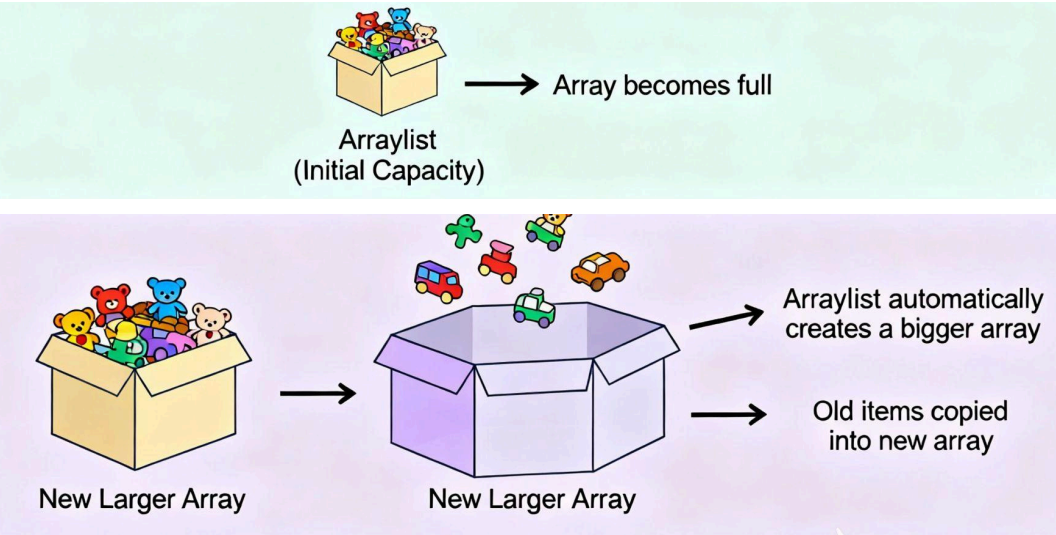
such as add, remove, search, and size so users do not need to create these functions manually.

Another key characteristic is that ArrayList supports **generics**, which means we can decide what type of data it will store (such as String, Integer, etc.). This helps prevent errors because we cannot insert the wrong type of data. The ArrayList also manages memory automatically. When more elements are added and the storage is full, it increases its capacity on its own. Users can also set an initial capacity to use memory more efficiently.

Feature	Array	ArrayList
Size	Fixed size (cannot change)	Flexible size
Type of Data	Can store primitives and objects	Can only store objects (not primitives directly)
Built-in Methods	Very few (length only)	Many methods (add, remove, get, set, size, etc.
Generics Support	No	Yes
Memory Handling	Manual, fixed	Automatic resizing

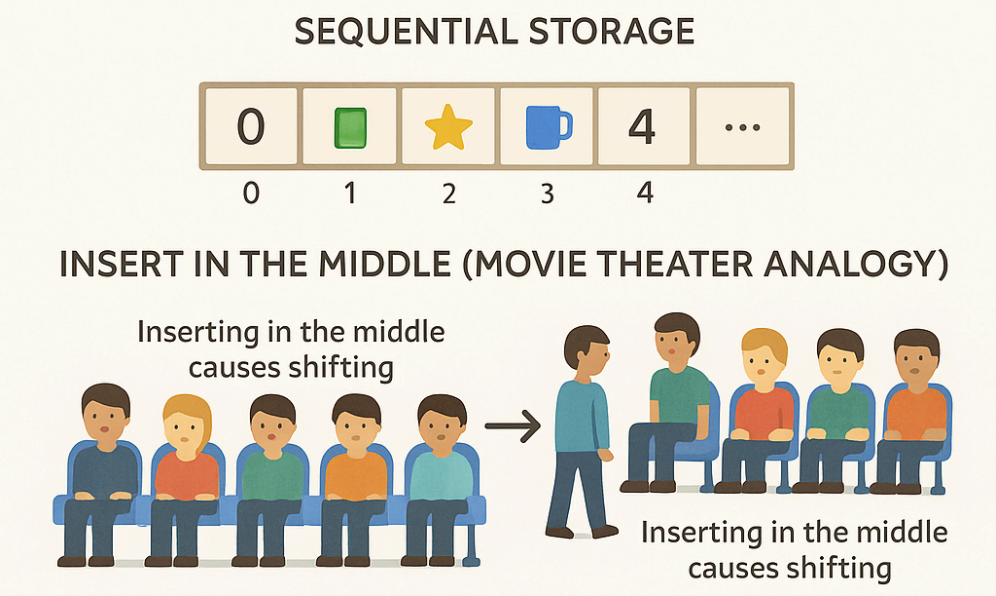
1.3. How ArrayList Works Internally

In Java, an **ArrayList** stores its data using a normal array. But unlike a regular array, when the storage becomes full, the ArrayList doesn't stop or give an error. Instead, it automatically creates a **bigger array** and copies all the old data into it. This happens quietly in the background, so you can keep adding items without worrying about the size.



Imagine a **box of toys**. You keep adding toys until the box is full. When it's full, you get a bigger box, move all the toys into it, and then add the new toy. This moving process takes some time, but it doesn't happen often.

The ArrayList also keeps all elements in **order**. So if you insert something in the middle, the other items must **shift** to make space just like people moving slightly in a movie theater when someone wants to sit in the middle. That's why inserting or deleting in the middle is slower, while accessing items by index is very fast.



1.4. Important ArrayList Methods

To help you better understand the various operations that can be performed on an **ArrayList**, the following table summarizes the most commonly used methods. It also includes brief explanations and simple analogies to make each method easier to grasp.

Method	Function	Analogy
`add(element)`	Adds an element to the end	Placing a new book at the end of a bookshelf
`add(index, element)`	Inserts an element at a specific position	Slipping a book into the middle of a shelf, causing other books to shift
`get(index)`	Retrieves an element by position	Picking a book from the shelf based on its slot number
`set(index, element)`	Replaces an element at a	Swapping an old book with

	specific position	a new one in the same spot
`remove(index)`	Removes an element at a specific position	Pulling a book from the middle of a shelf and other books move forward
`size()`	Returns the number of elements	Counting how many books are on the shelf
`isEmpty()`	Checks whether the list is empty	Checking whether the shelf is completely empty
`clear()`	Removes all elements	Taking all books off the shelf until it is completely empty

1.5. Example Use Cases of ArrayList

Study Case : In a train ticket booking system, passenger data must be managed quickly and neatly. Whenever a passenger books a seat, the system needs to add a new entry to the list. Sometimes there are priority passengers (such as the elderly or VIPs) who must be placed in a specific order. In addition, passengers may cancel their tickets, and the admin needs to update the data, check the total number of passengers who have booked, or clear the list after the departure schedule is completed. To handle all these needs, the **ArrayList** provides various methods that are very useful for managing the list of train passengers. The following explains each method along with examples in the context of train ticket booking.

1) **Declaration ArrayList**

   Creates a new ArrayList named buyers, which is used to store the list of customers or train ticket passengers. Explanation of each part:

- **`ArrayList<String>`** means that this list is only allowed to contain data of type String. In this case, each String contains passenger information such as: "Andi - Car 1 Seat 12A"
- **`buyers`** this is the variable name. It represents the main list where all passenger data is stored.
- **`new ArrayList<>()`** creates a new, empty ArrayList object.

```
ArrayList<String> buyers = new ArrayList<>();
```

```java
// Helper method to display the list of buyers
private static void displayList(ArrayList<String> buyers) {
    if (buyers.isEmpty()) {
        System.out.println("The buyer list is empty.");
    } else {
        System.out.println("Current Buyer List:");
        for (int i = 0; i < buyers.size(); i++) {
            System.out.println((i + 1) + ". " + buyers.get(i));
        }
    }
}
```

2) **`add(element)`**

This method is used to add a new passenger to the end of the booking list.

Example case: When someone books a ticket for a certain car and seat, their data is added as a new entry.

```java
// 1. add(element) - Add new buyers
System.out.println("1. Adding new buyers using add(element)");
buyers.add("DIGGIE - Car 1 Seat 12A");
buyers.add("IXIA - Car 2 Seat 7C");
buyers.add("LING - Car 3 Seat 5B");
displayList(buyers);

```

3) **`add(index, element)`**

This method inserts a passenger at a specific position, shifting the others down.**Example case:** A senior passenger or VIP needs to be placed at the top of the processing list for priority handling.

```java
// 2. add(index, element) - Insert priority buyers (senior/VIP)
System.out.println("\n2. Inserting a priority buyer at the first position using add(index, element)");
buyers.add(0, "Priority - XAVIER - Car 1 Seat 1A");
displayList(buyers);
```

4) **`get(index)`**

Used to get the data of a passenger in a specific position in the booking list.

Example case: The train staff needs to view the details of the 3rd passenger for verification.

```java
// 3. get(index) - Retrieve a specific buyer
        System.out.println("\n3. Retrieving the 3rd buyer using get(index)");
        String thirdBuyer = buyers.get(2);
        System.out.println("3rd Buyer: " + thirdBuyer);

```

5) **`set(index,element)`**

Replaces the passenger information at a certain position without changing the list size.**Example case:** A passenger requests to change seats (e.g., moving from Car 2 Seat 7C to Car 2 Seat 9D).

```java
// 4. set(index, element) - Update buyer information
        System.out.println("\n4. Updating the 2nd buyer using set(index, element)");
        buyers.set(1, "AMON - Car 2 Seat 9D (Changed Seat)");
        displayList(buyers);
```

6) **`remove(index)`**

Deletes a passenger at a specific position, and all passengers below that position shift up.

Example case: A passenger cancels their train ticket, so their entry is removed from the booking list.

```java
// 5. remove(index) - Remove a buyer who cancels
       System.out.println("\n5. Removing the 3rd buyer (ticket canceled) using remove(index)");
       buyers.remove(2);
       displayList(buyers);
```

7) **`size()`**

Returns the number of passengers currently in the booking list.

Example case: To check how many passengers have booked seats before departure.

```java

// 6. size() - Count total buyers
System.out.println("\n6. Checking the total number of buyers using size()");
int total = buyers.size();
System.out.println("Total buyers who booked: " + total);
```

8) **`isEmpty()`**

Used to check whether the list contains any passengers.

Example case: Before bookings open, the list will be empty, so staff can check if any early entries were made.

```java
// 7. isEmpty() - Check if the list is empty
    System.out.println("\n7. Checking if the buyer list is empty using isEmpty()");
    if (buyers.isEmpty()) {
        System.out.println("No buyers have booked tickets yet.");
    } else {
        System.out.println("The buyer list has entries.");
    }
```

9) **`clear()`**

Removes all passenger data from the booking list.

Example case: After the train has departed, the booking list for that trip is cleared.

```java
// 8. clear() - Clear the list after train departure
    System.out.println("\n8. Clearing the list after the train has departed using clear()");
    buyers.clear();
    displayList(buyers);
    System.out.println("isEmpty() after clear: " + buyers.isEmpty());
```

2. Linked List

2.1. Definition of LinkedList

A **LinkedList** is a data structure made of a chain of elements connected by links. Unlike an ArrayList, which stores data in one continuous block of memory, a LinkedList stores each element in separate memory locations. Each element is called a **node**, and every node has two parts: the **data** and a **pointer** to the next node.

You can imagine a LinkedList like a **train**. Each carriage (node) carries passengers (data) and is connected to the next carriage. If you want to add or remove a carriage, you just change the connections you don't need to move all the other carriages. This makes adding or removing elements very fast, especially at the beginning or in the middle of the list.

2.2. Basic Structure of a LinkedList

Before understanding how a LinkedList works as a whole, it's important to first learn about the main components that make up this structure.

1) **Node**

A **node** is the basic element that forms a **LinkedList**. Each node contains two parts: the **data** you want to store and a **pointer** that refers to another node. In simple terms, a node can be located anywhere in memory, but the pointer keeps all the nodes connected.

Analogy: A node is like a **train carriage**. Each carriage has content (data) and a connector that links it to the next carriage (pointer).

2) **Data**

The **data** is the main information stored inside a **node**. This value can be a number, text, an object, or any other data type. Each node can hold different data, and the data does not need to be stored in nearby memory locations.

Analogy: The data is like the **passenger or cargo** inside a train carriage.

3) **Next & Previous Point**

The **next** points to the **next node** in the LinkedList. This pointer allows the list to be traversed from the beginning to the end. If the next pointer is **null**, it means the node is the **last node** in the list.

Analogy: The next pointer is like the **front connector** of a train carriage that links it to the next carriage.

The **previous** points to the **previous node** and is used in a **Doubly Linked List**. With this pointer, the list can be traversed **forward and backward**, making navigation more flexible.

Analogy: The previous pointer is like the **rear connector** of a train carriage, allowing movement in the opposite direction.

4) **Pointer**

A **pointer** is a reference that connects one **node** to the next (or previous) node. The pointer creates the chain that forms a **LinkedList**, allowing the data to be traversed.

Analogy: A pointer is like the **connector between train carriages**, keeping all the carriages linked even when they are not placed tightly together.

5) **Head**

The **head** is the first **node** in a LinkedList. All operations such as traversing, searching, or counting nodes—always start from the head.

Analogy: The head is like the **front carriage** of a train, where you begin checking the entire train from the very front.

6) **Tail**

The **tail** is the **last node** in a LinkedList. In a normal (linear) list, the tail's **next pointer** is `null`. In a **Circular LinkedList**, however, the tail connects back to the **head**, forming a loop.

Analogy: The tail is like the **last carriage** of a train, and in the circular version, this last carriage is connected back to the **front carriage**.

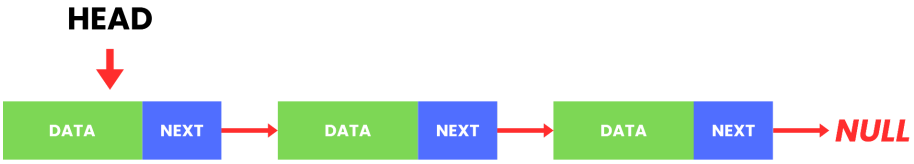
2.3. Types of LinkedList

LinkedList comes in several variations, each designed with different pointer structures and behaviors. Although all of them store data in the form of connected nodes, the way each node links to others determines how the list operates. Understanding these types helps developers choose the right LinkedList for insertion, deletion, traversal, or circular data management.

1) Singly Linked List

A **Singly Linked List** is the simplest type of **LinkedList**. Each **node** has only **one pointer**, and it points to the **next node**. This allows the list to be read from the **first node** to the **last node** in one direction.

Because it moves only forward, operations like **inserting** or **deleting** a node at the **beginning** of the list are very fast. However, a Singly Linked List **cannot go backward** because it has no pointer to the previous node.



2) Doubly Linked List

A **Doubly Linked List** is a more flexible version of a LinkedList because each **node** has **two pointers**: one pointing to the **previous node** and one to the **next node**. These

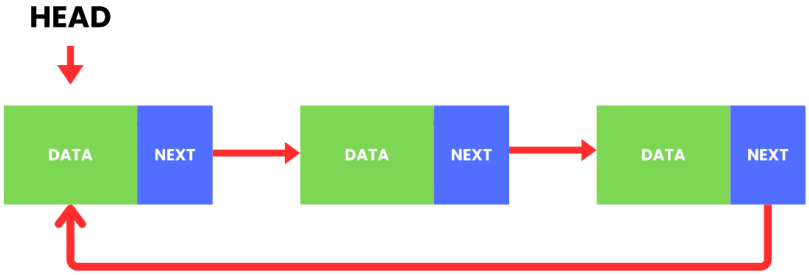
two directions allow the list to be **traversed forward and backward**, making it easier to search for or delete specific elements.

However, this two-way ability requires **more memory**, since each node must store an extra pointer. A Doubly Linked List is very useful when an application needs **two-way navigation**, such as in **playlist navigation**, **undo redo systems**, or any process that needs **back-and-forth movement** through data.

3) **Singly Circular Linked List**

A **Singly Circular Linked List** has a structure similar to a **Singly Linked List**, but the **last node** does not point to Null. Instead, it points back to the **first node**, forming a **circle**. Because of this circular structure, the list can be **traversed continuously** without reaching an end, making it useful for systems that need an **endless loop**, such as **queues** or **turn rotations**.

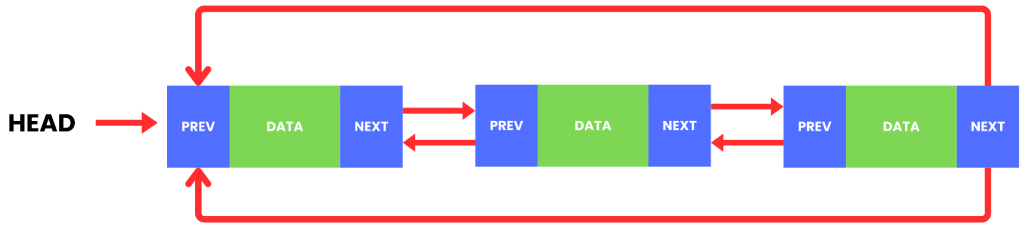
Even though the traversal is still **one-way**, the circular nature ensures that the list can always return to the beginning without checking for the end.



4) **Doubly Circular Linked List**

A **Doubly Circular Linked List** is the most complete form of all LinkedList types. Each **node** has **two pointers** one to the **previous node** and one to the **next node**. However, unlike a regular Doubly Linked List, the **last node** in this structure points back to the **first node**, and the **first node** points back to the **last node**, forming a **bidirectional circular structure**.

This circular, two-direction structure allows **unlimited forward and backward movement**, making it ideal for applications that need **continuous circular navigation**, such as **menu carousels**, **player rotations in games**, or **rotating queue simulations**.
However, this type uses the **most memory**, since each node stores two pointers and the list forms a circular connection.



2.4. Important LinkedList Operations

A **LinkedList** provides several important operations that allow us to **add**, **remove**, and **traverse** data flexibly.

1) Insertion

Insertion is the operation of adding a new **node** to a LinkedList. A node can be added at the **beginning**, **middle**, or **end** of the list. In a LinkedList, insertion is very efficient because it only requires updating **pointers**.

1. **Insert at Head:** Add the new node, point it to the old head, and make the new node the **new head**.
2. **Insert in the Middle:** Point the new node to the next node, and update the previous node's pointer to link to the new node.
3. **Insert at Tail:** Point the last node to the new node, then set the new node as the **new tail**.

Analogy: Adding a new train carriage is easy just attach it at the front, in the middle, or at the end without moving all the other carriages.

2) Deletion

Deletion is the operation of removing a **node** from a LinkedList. This can be done at the **beginning**, **middle**, or **end** of the list by adjusting the **pointers** so they skip over the node that is being removed.

- **Delete Head:** Move the **head** to the next node.
- **Delete Middle:** Connect the previous node directly to the node after the one being removed.
- **Delete Tail:** Update the pointer of the node before the tail so it points to **null**.

Analogy: Removing a train carriage is simple you just disconnect it and link the surrounding carriages together again, without rebuilding the entire train.

3) Traversal

Traversal is the process of visiting each **node** in a LinkedList, starting from the **head** and continuing until the last node. Because a LinkedList does not support **direct**

(random) access, traversal must be done **one node at a time** by following the **next pointer**.

Analogy: To check all the train carriages from front to back, you must walk from the **first carriage** to the **next one**, and continue in order until the last carriage.

4) Searching

Searching is done by traversing the LinkedList from the **head** until you find the node that contains the desired value. Since a LinkedList does not have indexes like an ArrayList, the search must be done **linearly**, from the beginning to the end.

Analogy: Searching for a specific passenger in a train means you have to check each carriage one by one until you find them.

5) Updating

Updating is used to replace the **data** stored in a node without changing the structure of the LinkedList. You first use **traversal** to find the node you want to update, then simply change its data value.

Analogy: If you want to update the passenger list in a specific train carriage, you just open that carriage and change the information. No need to move or reconnect any of the carriages.

6) Count Nodes

Counting the number of **nodes** is done by performing a **traversal** from the **head** to the end while increasing a counter for each node visited.

Analogy: To find out how many train carriages there are, you need to walk through the entire train, moving from one carriage to the next and counting them one by one.

2.5. Example Use Cases of LinkedList

Case Study : Managing Train Passenger Queues Using a LinkedList. In a train ticketing and boarding system, passenger data often changes dynamically. Some passengers arrive early, some enter with special priority, some cancel their trip, and sometimes staff need to rearrange the order of passengers for ticket inspection.

Class `Node.Java`

```java
class Node {
    String name;
    Node next;

    public Node(String name) {
        this.name = name;
        this.next = null;
    }
}
```

Class `TrainPassengerList.Java`

```java
class TrainPassengerList {

    Node head;
    Node tail;
```

1. `addPriorityPassenger`

```java
// Insert priority passenger (insert at head)
public void addPriorityPassenger(String name) {
    Node newNode = new Node(name);
    newNode.next = head;
    head = newNode;

    if (tail == null) {
        tail = newNode;
    }
}
```

2. addNormalPassenger

```java
// Insert normal passenger (insert at tail)
 public void addNormalPassenger(String name) {
     Node newNode = new Node(name);

     if (head == null) {
         head = newNode;
         tail = newNode;
         return;
     }

     tail.next = newNode;
     tail = newNode;
 }
```

3. addAfter

```java
// Insert after a certain passenger
   public void addAfter(String keyPassenger, String newPassenger) {
       Node current = head;

       while (current != null && !current.name.equals(keyPassenger)) {
           current = current.next;
       }

       if (current != null) {
           Node newNode = new Node(newPassenger);
           newNode.next = current.next;
           current.next = newNode;

           if (current == tail) {
               tail = newNode;
           }
       }
   }

```

4. addAfter

```java
// Remove passenger who cancels (delete by name)
public void removePassenger(String name) {
    if (head == null) return;

    if (head.name.equals(name)) {
        head = head.next;
        if (head == null) tail = null;
        return;
    }

    Node current = head;
    Node previous = null;

    while (current != null && !current.name.equals(name)) {
        previous = current;
        current = current.next;
    }

    if (current != null) {
        previous.next = current.next;
        if (current == tail) {
            tail = previous;
        }
    }
}
```

5. displayQueue

```java
// Traverse list
public void displayQueue() {
    Node current = head;
    System.out.println("Current Passenger Queue:");
    while (current != null) {
        System.out.println("- " + current.name);
        current = current.next;
    }
}
```

6. searchPassenger

```java
// Search passenger
public boolean searchPassenger(String name) {
    Node current = head;
    while (current != null) {
        if (current.name.equals(name)) return true;
        current = current.next;
    }
    return false;
}
```

7. updatePassenger

```java
// Update passenger data
public void updatePassenger(String oldName, String newName) {
    Node current = head;
    while (current != null) {
        if (current.name.equals(oldName)) {
            current.name = newName;
            return;
        }
        current = current.next;
    }
}
```

8. countPassengers

```java
// Count passenger
public int countPassengers() {
    int count = 0;
    Node current = head;
    while (current != null) {
        count++;
        current = current.next;
    }
    return count;
}
```

9. Class `TrainPassenger.java`

```java
private TrainPassengerList passengerList;

    public TrainPassenger() {
        this.passengerList = new TrainPassengerList();
    }

    // Delegating methods to the TrainPassengerList
    public void addNormalPassenger(String name) {
        passengerList.addNormalPassenger(name);
    }

    public void addPriorityPassenger(String name) {
        passengerList.addPriorityPassenger(name);
    }

    public void displayQueue() {
        passengerList.displayQueue();
    }

    public void addAfter(String keyPassenger, String newPassenger) {
        passengerList.addAfter(keyPassenger, newPassenger);
    }

    public void removePassenger(String name) {
        passengerList.removePassenger(name);
    }

    public boolean searchPassenger(String name) {
        return passengerList.searchPassenger(name);
    }

    public void updatePassenger(String oldName, String newName) {
        passengerList.updatePassenger(oldName, newName);
    }

    public int countPassengers() {
        return passengerList.countPassengers();
    }

```

```java
public static void main(String[] args) {

        TrainPassenger queue = new TrainPassenger();

        System.out.println("=== TRAIN PASSENGER QUEUE SIMULATION USING LINKEDLIST ===\n");

        System.out.println("Adding normal passengers...");
        queue.addNormalPassenger("GRANGER");
        queue.addNormalPassenger("OBSIDIA");
        queue.addNormalPassenger("BRUNO");
        queue.displayQueue();

        System.out.println("\nAdding PRIORITY passenger: 'MIYA'...");
        queue.addPriorityPassenger("MIYA");
        queue.displayQueue();

        System.out.println("\nInserting 'Family Group' after 'MIYA'...");
        queue.addAfter("MIYA", "Family Group");
        queue.displayQueue();

        System.out.println("\nRemoving passenger 'OBSIDIA' (canceled)...");
        queue.removePassenger("OBSIDIA");
        queue.displayQueue();

        System.out.println("\nSearching for passenger 'BRUNO'...");
        System.out.println(queue.searchPassenger("BRUNO") ? "Found" : "Not Found");

        System.out.println("\nUpdating passenger 'Family Group'...");
        queue.updatePassenger("Family Group", "Family Group (Updated)");
        queue.displayQueue();

        System.out.println("\nTotal passengers in queue: " + queue.countPassengers());
    }
```

CODELAB

Mission 1: Jujutsu Kaisen - ArrayList

- Objective: Help Gojo Satoru manage the student roster for Tokyo Jujutsu High using an `ArrayList`. The list must be dynamic to handle new students, transfers, and... unfortunate incidents
- Your Tasks:
  1. Declaration: Initialize the ArrayList to store the names of sorcerers.
  2. Recruitment: Use `.add()` to recruit Itadori, Fushiguro, and Kugisaki.
  3. Transfer Student: Use `.add(index, element)` to bring in Yuta Okkotsu.
  4. Identification: Use `.get(index)` to check who is currently at a specific rank.
  5. Possession: Use `.set(index, element)` to update Itadori's status when Sukuna takes over.
  6. Casualties: Use `.remove(index)` to update the roster after the Shibuya Incident. Status
  7. Report: Use `.size()` and `.isEmpty()` to check the strength of the school.
  8. Total Wipeout: Use `.clear()` for the Culling Game scenario.
- Output:

```
First Years Assembled: [Itadori, Fushiguro, Kugisaki]
After Yuta joins: [Okkotsu, Itadori, Fushiguro, Kugisaki]
Student at index 2 is: Fushiguro
Oh no, Itadori switched!: [Okkotsu, Ryomen Sukuna, Fushiguro, Kugisaki]
After the Shibuya Incident: [Okkotsu, Ryomen Sukuna, Fushiguro]
Remaining students: 3
The fight continues!
Post-Culling Game Status: []
```

Mission 2: One Piece - LinkedList File:

- Objective: Help Nami navigate the Grand Line! You must build a LinkedList that represents the chain of islands the Straw Hat crew visits. If the Log Pose link is broken, they will be lost at sea!
- Your Tasks:
  1. Map Structure: Complete the Island class (Node) constructor. Chart Course: Implement `addIsland()` to link a new island to the end of the chain.
  2. Buster Call: Implement `busterCall()` to delete an island from the map (re-link the previous island to the next one).
  3. Logbook: Implement the loop in `printLogbook`() to traverse and print the route. Navigation

4. Check: Implement `isIslandOnRoute`() to search if a specific island exists in the chain.
5. Adventure Count: Implement `countIslands`() to tally up the journey.

- Output:

```
Grand Line Route: Alabasta -> Skypiea -> Water 7 -> Enies Lobby (End)
Visited Fishman Island? false
Buster Call initiated on Enies Lobby!
Grand Line Route: Alabasta -> Skypiea -> Water 7 (End)
Total Islands visited: 3
```

You can clone the project from this GitHub repository, specifically from the 'modul2' branch
" **git clone -b modul2** https://github.com/dikameo/DATA-STRUCTURE.git "

DEMO

TASK 1 Passenger Management System (ArrayList)

Objective Develop a Java application to manage passenger records for a transportation system. Since the system requires frequent access to data by index (random access) and involves simple additions to the end of the list, you are required to use an **`ArrayList`** as the primary data structure.

Detailed Requirements

1. Class `Passenger`

Create a class named `Passenger` to represent an individual customer.

- **Attributes (Private Fields):**
  - `name` (String): The full name of the passenger.
  - `email` (String): The email address of the passenger.
  - `phone` (String): The contact number of the passenger.
  - `passengerId` (int): A unique identifier for the passenger. **Note:** This ID must auto-increment automatically whenever a new object is created.
  - `age` (int): The age of the passenger.
- **Methods:**
  - **Constructor:** Initialize the fields when a new object is created.
  - **Getters and Setters:** Provide public access to modify and retrieve private attributes.
  - `displayInfo()`: A method to print the passenger's details in a formatted, readable manner.
  - `toString()`: Override the standard method to return a string representation of the object.

2. Class `PassengerManager`

Create a manager class that handles the logic and storage of the application using `ArrayList<Passenger>`.

- **Data Structure:**
  1. `private ArrayList<Passenger> passengers;`
- **Required Operations:**
  1. **Add Passenger:** Create a method to append a new `Passenger` object to the end of the list.

2. **Get Passenger:** Create a method to retrieve a specific passenger object based on its index in the list.
3. **Search:** Implement a **Linear Search** algorithm to find passengers by their name.
4. **Update:** Create a method to modify the details of an existing passenger at a specific index.
5. **Remove:** Implement functionality to delete a passenger from the list. You should support removal by **Index** or by **Passenger ID**.
6. **Display All:** Create a method to display a complete list or table of all registered passengers currently in the system.

Task 2: Train Schedule System (LinkedList)

Objective Develop a system to manage train schedules. Train schedules can be viewed as a sequence or queue where the system often needs to efficiently add or remove schedules from the **beginning** (e.g., for immediate departures) or the **end** (e.g., for future schedules). Consequently, **`LinkedList`** is the chosen data structure due to its efficient **O(1)** performance for additions and removals at both ends.

Detailed Requirements

1. Class `Schedule`

Create a class named `Schedule` to represent a specific train trip.

- **Attributes:**
  - `scheduleId` (int): A unique identifier for the schedule. **Note:** This must be an **Auto-incremented** unique ID.
  - `trainCode` (String): The code of the train (e.g., "TRN001").
  - `trainName` (String): The name of the train service.
  - `origin` (String): The starting station.
  - `destination` (String): The arrival station.
  - `departureTime` (String): The time of departure.
  - `baseFare` (double): The cost of the ticket.
- **Methods:**
  - **Constructor:** Initialize all fields.
  - **Getters and Setters:** Provide access to the attributes.
  - `displayInfo()`: A method to print the schedule details in a specific format suitable for schedule tables.

2. Class `ScheduleManager`

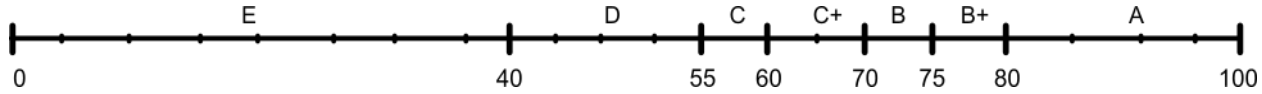
Create a manager class that utilizes `LinkedList<Schedule>` to store and manipulate the data.

- **Data Structure:**
  1. `private LinkedList<Schedule> schedules;`
- **Required Operations:**
  1. **Add First/Last:** Implement methods to add new schedules specifically at the beginning (`addFirst`) and at the end (`addLast`) of the list.
  2. **Remove First/Last:** Implement methods to remove schedules from the beginning (`removeFirst`) and from the end (`removeLast`).
  3. **Search Routes:** Implement a search function to find schedules based on their **Origin** or **Destination**.
  4. **Iterator Traversal:** Use an `Iterator` to traverse and display the list. This is required to demonstrate proper `LinkedList` iteration techniques.
  5. **Cleanup:** Implement a method to remove inactive or specific schedules efficiently using the `Iterator` (specifically using the iterator's `remove()` method to avoid `ConcurrentModificationException`).

Grading Rubric

Assessment Aspect	Points (Total 100%)
Codelab	**Total 10%**
Successfully run	10%
Demo	**Total 90%**
Task 1	**45%**
Implement Arraylist	5%
Implement Arraylist Operations	5%
Running Smoothly	10%
Answer Questions	25%
Task 2	**45%**
Implement LinkedList	5%
Implement Linkedlist Operations	5%
Running Smoothly	10%
Answer Questions	25%

Grading Scale



A = (81 - 100) → GOAT

B+ = (75 - 80) → Very Good

B = (70 - 74) → Good

C+ = (60 - 69) → Fairly Good

C = (55 - 59) → Fair

D = (41 - 54) → Poor

E = (0 - 40) → Bro really...